Course Code: ITA1439 **Course Name: Ethical Hacking** **Slot: A**

CO2 – ASSIGNMENT_2

Nature of Assignment: Individual **Duration:** 1 Week **Total Marks:** 50

Student Name: Sayed Fazal **Reg. No.:** 192311291

CO2: Execute footprinting and social-engineering techniques such as DNS analysis, competitive intelligence gathering and attack simulations to collect information for identifying potential security risks.

Assignment Title: Evaluation and Engineering of Footprinting, Reconnaissance and Social-Engineering-Awareness Workflows in an Authorized Assessment

Scenario - Assessment Design Philosophy

The assignment is designed for high-performing cybersecurity students and emphasizes the evaluation of reconnaissance and information-gathering choices rather than the mechanical use of tools. Students are expected to formulate an information-gathering strategy, construct difficult test conditions, run footprinting against authorized targets, collect evidence, identify limitations, compare alternatives and defend engineering decisions. All work is performed only against authorized or lab-owned targets, and no attack is launched against live systems or real individuals.

Mark Distribution and Coverage Map

Question	Assessed Area	Marks	Report Section
Q1	Footprinting strategy design and evaluation	8	Section 1
Q2	DNS footprinting and analysis workflow	7	Section 2
Q3	Competitive-intelligence and awareness utility	7	Section 3
Q4	Reconnaissance data validation and recovery	6	Section 4
Q5	Hierarchical organization footprinting	7	Section 5
Q6	Footprinting approach comparison	7	Section 6
Q7	Integrated diagnose–redesign–validate challenge	8	Section 7
	TOTAL	50	

Common Assessment Lifecycle

Every question follows the required cycle: Understand → Predict → Design → Collect → Analyse → Evaluate → Defend → Validate. Each section below states the problem, the prediction, the design, the collection plan, the analysis and the defence. Execution time is reported only as supporting evidence; coverage, source reliability and scope compliance are the primary evidence.

Q1 – Evaluate and Engineer a Footprinting Strategy 8 Marks

1.1 Understand

The team needs to footprint example-corp.example before a penetration test. A single source is incomplete and can mislead: a clean WHOIS record can still leak information through DNS, and a quiet DNS footprint can be exposed by public documents. The strategy must combine several sources, classify each technique, and judge the reliability of each source.

1.2 Source and technique classification

A technique is classed as **passive** if it queries third-party data stores (registry data, public resolvers, search indexes, web archives) and does not send traffic to the target's own infrastructure. It is classed as **active** if it sends queries directly to the target's authoritative servers or services.

Technique	Data source	Class	Reliability notes
WHOIS / RDAP lookup	Registry and registrar records	Passive	Authoritative for registration fields, but often redacted by privacy services and may be outdated.
Public recursive DNS (A, MX, NS, TXT)	Cached answers from a public resolver	Passive	Reflects cache state; TTL may hide recent changes. Good for breadth, not for proving current configuration.
Direct queries to authoritative NS	Target's own nameservers	Active	Most current and authoritative for the zone; leaves logs on target infrastructure.
Zone transfer (AXFR)	Target's nameserver	Active	Only where explicitly permitted in writing. Highest sensitivity.
Search-engine operators (site:, filetype:, intitle:)	Search index	Passive	Index is stale and may include third-party copies; verify each hit against the origin.
Archived pages	Web archive snapshots	Passive	Shows historical content; may contain removed or wrong data. Always check capture date.
Public documents (annual reports, filings)	Published PDFs and filings	Passive	Authoritative for what was published, not for current state.

1.3 Designed test targets

WHOIS and public DNS only return meaningful data for real registered names. For the lab, register four domains you own, or build a local authoritative zone (for example BIND or dnsmasq) and place the search-engine and archive tests on lab web pages. The four contrasting targets are:

- **lab-protected:** privacy-redacted WHOIS, DNS fronted by a managed provider, no AXFR, no sensitive TXT records.
- **lab-leaky:** clean WHOIS but a public document, archived page and job posting that reveal subdomains and vendors.
- **lab-misdns:** a lame or inconsistent NS delegation and an MX pointing at a stale host.
- **lab-minimal:** a single A record and no other public data, to test the empty-result path.

1.4 Prediction before collection

Target	Predicted WHOIS	Predicted DNS	Predicted search / archive	Predicted OBSERVED result
lab-protected	Redacted registrant; registrar only	A, MX, NS to provider; no TXT of note	No indexed internal docs	[OBSERVED]
lab-leaky	Registrant name visible	A, MX, NS; several CNAMEs to subdomains	Annual report lists vendor and subdomain	[OBSERVED]
lab-misdns	Visible registrant	NS delegation inconsistent; stale MX	Old archived page with old host	[OBSERVED]
lab-minimal	Redacted	Single A record	Nothing indexed	[OBSERVED]

1.5 Strategy design

1. Scope manifest: record the authorized domains, subdomains and third-party services before any query. Anything not on the manifest is tagged OUT-OF-SCOPE and not queried actively.
2. Passive phase first: WHOIS, public DNS, search operators and archives. This builds the candidate list at zero contact with target infrastructure.
3. Verification phase: direct authoritative queries only for names already on the manifest and only for the record types needed to confirm passive findings.
4. Reliability scoring: each item carries its source, collection time and a reliability grade (verified, single-source, or stale) before it enters the analysis.
5. Conflict log: where two sources disagree (for example public resolver A record vs authoritative A record), both are recorded and the authoritative answer is preferred for current state.

1.6 Source-reliability and contradiction analysis

Record the sources queried and the records found, then list contradictions. Use the template below. Each row must cite the query used so that the result can be reproduced.

Target	Source A and result	Source B and result	Contradiction?	Which source to trust and why
lab-misdns	Public resolver: MX = old host	Authoritative NS: MX = new host	[OBSERVED]	Authoritative (current zone state); resolver is cached
lab-leaky	Archive: subdomain present	Live query: subdomain NXDOMAIN	[OBSERVED]	Live authoritative answer; archive is historical
lab-protected	[OBSERVED]	[OBSERVED]	[OBSERVED]	[OBSERVED]

1.7 Changed condition: from one domain to fifty subdomains and third-party services

The original strategy works for one domain but does not scale as written. Three modifications are required:

- **Batching by zone:** group subdomains by parent zone so that each NS is queried once rather than once per host.

- **Caching of passive results:** a single WHOIS and public-resolver result per apex domain is reused across all subdomains, cutting repeated lookups.
- **Scope gate before every active query:** a manifest check runs first, so third-party services outside the authorized list are never probed. Third-party services are passive-only unless explicitly authorized.

Defence. The strategy remains suitable for large-scale authorized reconnaissance because the passive phase is cheap and broad, and the active phase is narrowed by the scope gate. Its weakness is that cached passive data can be stale, so the verification phase must always re-confirm any item that will be acted on.

Q2 – Build a Reliable DNS Footprinting Workflow 7 Marks

2.1 Understand

A DNS map needs A, MX, NS and TXT records and, only where permitted, a zone transfer. Records can be inconsistent because of split-horizon views, delegation to third parties, caching and misconfiguration. The workflow must interpret results rather than report them at face value.

2.2 Workflow design (dig and nslookup)

Commands below are for authorized lab zones only. Replace the placeholders with your lab names and addresses.

```
# Step 1: record the resolver and timestamp for reproducibility
date -u +%FT%TZ > run_log.txt

# Step 2: authoritative NS list (delegation view)
dig +noall +answer NS lab-leaky.test > ns.txt

# Step 3: query each authoritative NS directly for each record type
for NS in $(awk '{print $5}' ns.txt | sed 's/\.$//'); do
  for T in A MX NS TXT; do
    dig @$NS lab-leaky.test $T +noall +answer >> records_$NS.txt
  done
done

# Step 4: cross-check with nslookup for the same names
nslookup -type=MX lab-leaky.test <lab-ns-ip>

# Step 5: zone transfer ONLY where written authorization exists
# dig @<lab-ns-ip> lab-leaky.test AXFR
```

2.3 Six contrasting DNS cases with predictions

Case	Configuration	Predicted exposed records	Predicted AXFR	OBSERVED
C1 Correctly secured	Restricted recursion, allow-transfer none	A, MX, NS only	Refused	[OBSERVED]
C2 Permits zone transfer	allow-transfer any (lab only)	Full zone including internal hosts	Succeeds	[OBSERVED]
C3 Stale records	Old A record, long TTL, new IP live	Old A in caches; new A authoritative	Refused	[OBSERVED]
C4 Split-horizon	Internal view vs external view	Only external view visible	Refused on external	[OBSERVED]
C5 Delegation mismatch	Parent NS differs from child NS	Inconsistent NS across queries	Refused	[OBSERVED]
C6 Minimal footprint	Only A and NS, no TXT	A, NS; MX and TXT empty	Refused	[OBSERVED]

2.4 Record analysis and prediction-versus-observation

For each case, compare the observed record set with the predicted set. Mark each record as MATCH, EXTRA (observed but not predicted) or MISSING (predicted but not observed), and

state the cause. Cases where an external and an internal view differ must be noted as split-horizon effects, not as errors.

Case	Predicted count	Observed count	MATCH	EXTRA	MISSING	Cause
C1	[pred]	[OBS]	[OBS]	[OBS]	[OBS]	[OBS]
C2	[pred]	[OBS]	[OBS]	[OBS]	[OBS]	[OBS]
C3	[pred]	[OBS]	[OBS]	[OBS]	[OBS]	[OBS]
C4	[pred]	[OBS]	[OBS]	[OBS]	[OBS]	[OBS]
C5	[pred]	[OBS]	[OBS]	[OBS]	[OBS]	[OBS]
C6	[pred]	[OBS]	[OBS]	[OBS]	[OBS]	[OBS]

2.5 Zone-transfer risk evaluation

A successful transfer (C2) exposes every host name in the zone, including internal names. The question is whether this changes the attacker's knowledge. Use this reasoning in your report:

- If the zone already contains only names discoverable from public DNS, a transfer adds little new knowledge but still confirms the full host list and internal naming scheme, which speeds up targeting.
- If the zone contains internal-only names, hostnames indicating roles (for example admin, backup, vpn-old), or unpublished subdomains, the transfer materially increases knowledge and raises the resulting risk.

Rate the risk for C2 as Low, Medium or High, with the reason tied to the observed record set.

2.6 Countermeasure and redesigned analysis

Two countermeasures are introduced: restricting transfers to named secondaries, and requiring TSIG-signed transfer requests. Example BIND configuration for the lab zone:

```
key "xfer-key" {
    algorithm hmac-sha256;
    secret "<lab-generated-secret>";
};

zone "lab-leaky.test" {
    type master;
    file "/etc/bind/db.lab-leaky.test";
    allow-transfer { key "xfer-key"; };
};
```

Repeat the six cases under this configuration and re-run the zone-transfer attempt. The redesigned analysis should show the AXFR result change from Succeeds to Refused for C2, with the same passive and non-transfer records still visible. The reduced exposure is the difference between the two record sets.

Measure	Before countermeasure	After countermeasure
AXFR result (C2)	[OBSERVED]	[OBSERVED]
Number of names exposed	[OBSERVED]	[OBSERVED]
Internal-only names exposed	[OBSERVED]	[OBSERVED]

Measure	Before countermeasure	After countermeasure
Risk rating	[OBSERVED]	[OBSERVED]

Q3 – Create a Competitive-Intelligence Awareness Utility 7 Marks

3.1 Understand

The department needs a repeatable workflow that gathers public business information about an authorized organization, maps each item to a social-engineering technique it could enable, and recommends awareness countermeasures. No social-engineering attack is performed against real people. All analysis runs against the lab test organizations defined in 3.5.

3.2 Modular workflow architecture

The utility is built as four modules that pass a common record format. A new source can be added as one module without changing the others.

Module	Function	Output
M1 Source adapter	One documented procedure per public source (job ads, annual reports, press releases, public profiles)	Raw items with source URL and capture date
M2 Item extractor	Extracts the business fact (role, tool, vendor, location, event)	Normalized item: type, value, source
M3 Technique mapper	Maps each item to phishing, vishing, pretexting or tailgating and the indicator to notice	Item-to-technique record
M4 Awareness reporter	Assigns a human-risk level and a countermeasure	Risk table and countermeasure list

3.3 Source operators and documented steps

Operators are run against the lab test sites only. Example queries (replace domain names with the lab domains):

Source	Operator query	Documented step
Job advertisements	site:careers.lab-rich.test intitle:"engineer"	Record URL, capture date, listed tools and locations
Annual reports	site:lab-rich.test filetype:pdf "annual report"	Record URL, filing year, vendor names and office sites
Press releases	site:news.lab-rich.test intitle:"press release"	Record URL, date, events, new offices or partners
Public profiles	site:profiles.lab-rich.test "IT"	Record URL, role titles, reporting lines (no personal data collected)

Personal data is not stored. Items are recorded at the level of role, function and organization, which is sufficient to assess the risk.

3.4 Item-to-technique mapping

Collected item	Technique it could enable	Indicator a target should notice
Job ad listing internal tool names and VPN product	Phishing (credible lure) and pretexting	Unsolicited emails referencing that tool or a 'required update'

Collected item	Technique it could enable	Indicator a target should notice
Org chart implied by titles and reporting lines	Vishing (a caller who knows names and roles)	Caller requests an action from a role they do not hold, or cannot confirm an internal extension
Press release announcing an office move or badge change	Tailgating	Unknown person near a secure entrance citing the new building
Annual report naming vendors and partners	Pretexting (posing as a vendor)	Invoice or account change from a known vendor with new bank details

3.5 Test organizations

Test org	Public information profile	Expected human-risk level
Rich (lab-rich.test)	Many job ads, reports, press releases with tools, vendors, offices	High
Sparse (lab-sparse.test)	One annual report and no job ads	Low
Misleading (lab-misleading.test)	Outdated press release naming a vendor that has since changed; one fake-looking profile	Medium, with false positives

3.6 Prediction-versus-observation and most dangerous source

Test org	Source	Predicted risk	Predicted items	OBSERVED items	OBSERVED risk
Rich	Job ads	High	[pred]	[OBS]	[OBS]
Rich	Annual report	High	[pred]	[OBS]	[OBS]
Rich	Press release	Medium	[pred]	[OBS]	[OBS]
Sparse	Annual report	Low	[pred]	[OBS]	[OBS]
Misleading	Press release (outdated)	Medium	[pred]	[OBS]	[OBS]

Evaluation. Decide whether the collected information meaningfully increases social-engineering risk by comparing the rich and sparse results. Identify the most dangerous source as the one that yields the most actionable items per query, usually job advertisements because they reveal tools and roles together. Support the claim with the OBSERVED counts.

3.7 Adding two new sources or countermeasures

Two additions are made without changing the module structure: a **new source** (an employee professional-profile module, added as M1 adapter) and a **new countermeasure** (a call-back verification rule for vendor bank changes). Evaluate the effect by re-running the rich test organization and recording the change in item count and risk rating.

Measure	Before additions	After additions
Items found (rich org)	[OBS]	[OBS]
High-risk items	[OBS]	[OBS]
Countermeasures recommended	[OBS]	[OBS]

3.8 Recommended awareness countermeasures

- Remove or generalize internal tool names in public job advertisements.
- Require call-back verification on any vendor bank-detail change, using a number held on file.
- Brief reception and facilities staff on verifying visitors before tailgating is possible.
- Review press releases for office and access details before publication.

Q4 – Design a Reconnaissance Data-Validation Mechanism 6 Marks

4.1 Understand

Reconnaissance data may be incomplete, outdated, duplicated or out of scope. Acting on poor data wastes effort and can breach scope, so each item is validated before use. The validator must distinguish items that can be recovered by re-query from items that must be rejected.

4.2 Validation design and classes

Each item is checked for four properties: scope, freshness, duplication and source reliability. Each item leaves the validator as one of three outcomes:

- **PASS:** all four checks succeed and the item may be used.
- **RECHECK (recoverable):** the item is in scope but stale, or from a source with a single unverified grade. It needs a re-query or confirmation.
- **REJECT (unrecoverable):** the item is out of scope, unverifiable, or a duplicate of an item already accepted.

```
function validate(item, manifest, seen, now):
    if item.host not in manifest:              return REJECT  # scope
    if hash(item) in seen:                     return REJECT  # duplicate
    if item.source_grade == 'unverified':      return REJECT
    if now - item.captured > MAX_AGE:          return RECHECK # freshness
    if item.source_grade == 'single-source':   return RECHECK # reliability
    seen.add(hash(item))
    return PASS
```

4.3 Adversarial test sequences

Five test sequences are constructed. Each sequence contains items with known properties so that the expected outcome is set before running the validator.

Sequence	Composition (10 items)	Predicted PASS	Predicted RECHECK	Predicted REJECT	OBSERVED P / R / X
S1 All valid	10 in scope, fresh, verified, unique	10	0	0	[OBS]
S2 Out of scope	6 valid + 4 out-of-scope hosts	6	0	4	[OBS]
S3 Stale	5 valid + 5 older than MAX_AGE	5	5	0	[OBS]
S4 Duplicated	6 unique + 4 exact duplicates	6	0	4	[OBS]
S5 Mixed adversarial	3 valid, 2 stale, 2 out-of-scope, 2 duplicates, 1 unverified	3	2	5	[OBS]

4.4 Ordering evaluation

The checks have different costs. Scope lookup against the manifest is a local, in-memory comparison. Duplicate detection is a hash lookup. Freshness is a timestamp comparison. Reliability is a grade lookup. Confirmation re-query is a network action and is the most expensive. The order matters in two ways: cheap checks that reject many items early reduce

work, and placing scope first prevents out-of-scope data from ever being re-queried, which would itself breach scope.

Ordering	Expected re-queries	Expected scope errors leaking through	Assessment
Original: freshness → reliability → scope → duplicate	High (stale items re-queried before scope check)	None, if re-query is post-scope	Wasteful; out-of-scope items can be re-queried
Redesigned: scope → duplicate → reliability → freshness	Low (only in-scope, unique, reliable-stale items)	None	Preferred; cheapest checks first, scope gate first

4.5 Redesigned order and justification

The redesigned order is scope first, then duplicate, then reliability, then freshness. Scope is first because it is the only check whose failure is a legal or ethical breach if ignored. Duplicates are second because they remove work before any grading. The re-query is only triggered for RECHECK items, so the expensive network action runs on the smallest set.

4.6 Scalability inquiry: one target to fifty targets

Assume each target yields 10 items, with the same mix as S5 (30 percent RECHECK, 50 percent REJECT, 20 percent PASS). The table predicts totals as the target count grows. Observed values come from running the validator on generated data for 1, 10 and 50 targets.

Targets	Items total	Predicted PASS	Predicted RECHECK	Predicted REJECT	Predicted re-queries	OBSERVED re-queries
1	10	2	3	5	3	[OBS]
10	100	20	30	50	30	[OBS]
50	500	100	150	250	150	[OBS]

Effort grows linearly with item count if the manifest is a set lookup. Duplicate detection grows with the size of the seen-set, so with 50 targets the hash set grows to hundreds of entries but remains cheap. The main scaling risk is the re-query queue, which grows with the RECHECK count; a per-host rate limit is needed to avoid detection.

Q5 – Evaluate Footprinting of a Hierarchical Organization 7 Marks

5.1 Understand

A target has a nested footprint of a parent domain, subdomains, departments and third-party services. Footprinting must traverse this hierarchy level by level, record each level, and stop cleanly at out-of-scope or terminal nodes.

5.2 Hierarchical representation

The footprint is modelled as a rooted tree. Each node has a type (domain, subdomain, department, service), a scope flag, and a list of children. Example shallow footprint:

```
example-corp.example  [domain, in-scope]
├── www             [subdomain, in-scope]
│   └── cdn.vendor.example  [service, OUT-OF-SCOPE]
├── mail            [subdomain, in-scope, terminal]
├── hr              [subdomain, in-scope]
│   └── payroll     [department, in-scope, terminal]
└── legacy          [subdomain, in-scope, no public data]  <- empty branch
```

5.3 Traversal design with explicit boundary condition

The traversal is breadth-first, so each level is fully recorded before the next. The boundary condition is explicit: a node is expanded only if it is in scope, not terminal, and has not been visited.

```
queue = [root]; visited = {}; record = []
while queue not empty:
    n = queue.pop_front()
    if n in visited:                continue  # no re-visits
    visited.add(n)
    if not n.in_scope:              record(n, 'BOUNDARY-STOP: out of scope');
continue
    record(n)
    if n.terminal or no children:   record(n, 'TERMINAL'); continue
    queue.extend(n.children)
```

5.4 Test structures

Footprint	Shape	Node count (predicted)	Depth (predicted)	Empty branch?	OBSERVED nodes visited
F1 Shallow	1 root, 4 subdomains, 8 services	13	2	No	[OBS]
F2 Deeply nested	Chain of 6 levels with one service at each	7	6	No	[OBS]
F3 Irregular	Branches of sizes 1, 5, 2, 0, 9	18	3	Yes: branch with 0 children	[OBS]
F4 Empty node	Root with one subdomain that has no public data	2	1	Yes	[OBS]

5.5 Prediction-versus-observation and boundary analysis

Compare the predicted node count and coverage for each footprint with the observed traversal log. For the empty branch (F3 and F4), state whether the boundary condition records the node as TERMINAL and whether it stops without error. A correct result is one where the empty node is recorded once, is not expanded, and does not stop the traversal of its siblings.

Footprint	Predicted coverage	OBSERVED coverage	Boundary handled correctly?	Notes
F1	13 of 13	[OBS]	[OBS]	[OBS]
F2	7 of 7	[OBS]	[OBS]	[OBS]
F3	18 of 18	[OBS]	[OBS]	[OBS]
F4	2 of 2	[OBS]	[OBS]	[OBS]

5.6 Effect of depth and branching

Depth increases the number of sequential steps and the time before the deepest level is reached, but not the total number of nodes unless branching also increases. Branching increases the node count and the number of queries per level. Completeness is affected by the boundary condition: a too-strict boundary (for example, rejecting all services) loses coverage, while a too-loose one leaves scope.

5.7 Systematic versus ad-hoc traversal

An ad-hoc approach follows whatever links or names are found next. It tends to revisit nodes, miss siblings of a branch that was followed deeply, and cross out of scope without noticing. The systematic traversal records each level, guarantees each node is visited once, and makes the scope decision explicit at every node. The cost is a manifest to maintain and a slightly slower start.

Q6 – Evaluate Footprinting Approaches Under Constraints 7 Marks

6.1 Approach definitions

- **Approach A – Passive open-source collection:** WHOIS, public DNS resolver, search operators and archives. Produces a broad candidate list with no direct contact with target infrastructure. Quiet, but slower to verify.
- **Approach B – Active DNS and service footprinting:** direct queries (A, MX, NS, TXT) to the target's authoritative servers and light service checks on in-scope hosts. Produces verified current state. Faster per host, but generates traffic that can be detected.

6.2 The two workflows

6. A: list the apex domain → WHOIS → public DNS for apex → search operators per subdomain candidate → archive lookups → deduplicate → output candidate list.
7. B: list the apex domain → authoritative NS lookup → for each in-scope host, query A, MX, TXT → confirm service banner on in-scope hosts only → output verified list.

6.3 Scope sizes and predicted queries

Four increasing scope sizes are tested against the same authorized lab target. Scope size is the number of in-scope hosts. Predicted query counts use the workflow steps above; A uses about 3 queries per host plus fixed overhead, and B uses 4 record queries per host plus NS lookups per zone.

Scope size (hosts)	Predicted A queries	Predicted B queries	Predicted A feasible?	Predicted B feasible?	OBSERVED A / B queries	OBSERVED time (supporting only)
1	5	7	Yes	Yes	[OBS]	[OBS]
5	17	23	Yes	Yes	[OBS]	[OBS]
15	47	63	Yes, slow	Yes	[OBS]	[OBS]
50	152	207	Slow, low completeness	Impractical: detection and rate limits	[OBS]	[OBS]

6.4 Redundancy analysis

Record where effort is wasted. In approach A, the same archive page is often fetched for several subdomains, and the apex public-DNS result is re-fetched per candidate. In approach B, the NS list is re-queried per host and the same TXT record is repeated for hosts sharing a zone. Count repeated items as follows:

Approach	Repeated item	Predicted repeats	OBSERVED repeats	Wasted queries (predicted / OBSERVED)
A	Apex WHOIS and public DNS	Once per candidate	[OBS]	[pred] / [OBS]
A	Archive pages shared across hosts	Several per shared page	[OBS]	[pred] / [OBS]

Approach	Repeated item	Predicted repeats	OBSERVED repeats	Wasted queries (predicted / OBSERVED)
B	NS lookup per host	Once per host	[OBS]	[pred] / [OBS]
B	Shared TXT records	Once per host sharing zone	[OBS]	[pred] / [OBS]

6.5 Faster or quieter: is either always better?

Neither is always better. Approach B is faster per host and gives verified current state, but at 50 hosts it generates detectable volume and may trigger rate limits, which invalidates the engagement if detection is a constraint. Approach A is quieter and cheap to scale, but its results are historical and need verification before use. The better choice depends on whether the engagement values detection avoidance, completeness, or freshness most.

6.6 Defended recommendation

- **Small-scope engagement (1 to 5 hosts):** Approach B, because the active queries are few, verification is immediate, and the detection risk is low. Passive results are used only as a cross-check.
- **Large-scope engagement (15 to 50 hosts):** A hybrid. Run Approach A first to build the candidate list cheaply and quietly, then run Approach B only against in-scope hosts from the list, batched by zone, so that the active volume stays low.
- **Teaching context:** Run both approaches so that students see the trade-off; real assessments follow the hybrid recommendation above.

Q7 – Integrated Review: Diagnose, Redesign and Defend 8 Marks

This section selects Q6's Approach B (active DNS footprinting) because its redundant collection is a measurable limitation with a clear fix. The workflow is diagnosed, a measurable criterion is set, a redesign is built, both versions are compared under equal conditions, and a final judgment is defended.

A. Diagnose the limitation (2 marks)

Bottleneck: repeated authoritative NS lookups and repeated record queries for hosts sharing a zone. The evidence is the redundancy table from Section 6.4 and the per-scope query counts from Section 6.3. The predicted repeats are at least one NS lookup per host and one TXT lookup per host sharing a zone. The observed repeat count from Section 6.4 confirms or refutes this.

Cause: redundant collection, not source unreliability and not poor ordering. The scope gate is already first, so the risk of leaving scope is low. The fix therefore targets redundancy.

B. Evaluation criteria (1 mark)

Criterion	Measure	Target direction
Coverage	Number of in-scope hosts with verified records / total in-scope hosts	Maintain or increase
Number of queries	Total DNS queries issued	Decrease
Source reliability	Share of records from authoritative NS (not cache)	Maintain at 100 percent for active records
Scope compliance	Out-of-scope queries issued	Must be 0
Time	Wall-clock time (supporting only)	Report only

C. Redesign and implementation (2 marks)

Design logic. Three changes remove the redundancy without weakening scope or reliability:

8. Query each zone's authoritative NS once and cache the list for the run.
9. Group hosts by zone; for each zone, query each record type once per unique name, not once per host that shares the name.
10. Keep the scope gate first, so no query is issued for an out-of-scope name.

```
redesigned_b(manifest, apex):
    ns_list = authoritative_ns(apex)            # once per zone
    zones = group_hosts_by_zone(manifest.hosts(apex))
    for zone, hosts in zones:
        for host in hosts:                      # scope gate first
            if host not in manifest: continue
        for name in unique(hosts):
            for T in [A, MX, TXT]:
                cache.get_or_query(name, T, ns_list)
    return verified(cache)
```

Trade-offs introduced. The cache adds code and memory. Caching the NS list across a long run risks using a stale delegation if the zone changes during the assessment, so the cache

lifetime is limited to a single run. Grouping means one failed query affects a group, so failures must be retried per name. These trade-offs are measured in Section D.

D. Validation and comparison (2 marks)

Both workflows are run under equivalent conditions: the same lab zone, the same host list, the same resolver, the same time window, and the same rate limit. Predicted values are recorded before running.

Measure (equal conditions)	Original B (predicted)	Original B (OBSERVED)	Redesigned B (predicted)	Redesigned B (OBSERVED)	Improved?
Total DNS queries (15 hosts)	63	[OBS]	about 30	[OBS]	[OBS]
Redundant queries	about 33	[OBS]	about 0	[OBS]	[OBS]
Coverage (verified / in-scope)	15 / 15	[OBS]	15 / 15	[OBS]	[OBS]
Out-of-scope queries	0	[OBS]	0	[OBS]	[OBS]
Authoritative share of records	100%	[OBS]	100%	[OBS]	[OBS]

The redesign is judged to improve the selected criterion (number of queries) only if the observed query count drops while coverage and scope compliance hold at their original values. Any drop in coverage, or any out-of-scope query, means the redesign failed regardless of query count.

E. Defended final engineering judgment (1 mark)

- **Did the redesign meet its objective?** Yes if the observed query reduction is confirmed in Section D and coverage and scope compliance are unchanged. State the measured figures.
- **When the original workflow is preferable:** for a small engagement of one or two hosts, the cache logic adds code and its cost outweighs the saving, and the original is simpler to audit and to explain in a viva. Also preferable when the assessor needs each query logged individually for evidence.
- **When the redesign is preferable:** for large scope, where redundant queries add detectable volume and time.
- **Defence rule:** the final choice rests on the measured figures in Section D, not on the expectation that caching is better in general.